📓 INTRODUCTION TO PYTHON (FULL VIVA NOTES)

🔷 1. Introduction to Python Programming

📖 Definition

Python is a **high-level, interpreted, general-purpose programming language** known for its **simple syntax and readability**.

💡 Key Points

- Developed by **Guido van Rossum (1991)**
- Uses **interpreter (no compilation required)**
- Code runs line by line
- Supports multiple paradigms:
  - Procedural
  - Object-Oriented
  - Functional

🧪 Example

```
print("Hello World")
```

❓ Viva Questions

👉 Q1: What is Python?
Ans: Python is a high-level, interpreted, general-purpose programming language.

👉 Q2: Who developed Python?
Ans: Guido van Rossum

👉 Q3: Why is Python called interpreted language?
Ans: Because code executes line by line without compilation.

👉 Q4: Is Python compiled or interpreted?
Ans: Interpreted

👉 Q5: What are programming paradigms supported?
Ans: Procedural, OOP, Functional

◈ 2. Features of Python

📖 Main Features

1. **Easy to Learn**
    - Simple syntax (like English)
2. **Interpreted Language**
    - No need to compile
3. **Dynamically Typed**
    - No need to declare data type
4. **Platform Independent**
    - Write once, run anywhere
5. **Object-Oriented**
    - Supports classes and objects
6. **Large Standard Library**
    - Many built-in modules

❓ Viva Questions

👉 Q1: What are features of Python?
Ans: Easy, interpreted, dynamic typing, platform independent, large libraries

👉 Q2: What is dynamic typing?
Ans: Data type is decided at runtime

👉 Q3: Why Python is platform independent?
Ans: Because it runs on interpreter

👉 Q4: What is interpreted language advantage?
Ans: Easy debugging

◈ 3. Applications of Python

📖 Where Python is Used

- Web Development → Flask, Django
- AI/ML → TensorFlow, Scikit-learn

- Cybersecurity → scripting, automation
- Data Science → Pandas, NumPy
- Game Development
- Automation & Scripting

❓ Viva Questions

👉 Q1: Where Python is used?
Ans: Web, AI, Data Science, Automation

👉 Q2: Why Python used in AI?
Ans: Due to strong libraries

👉 Q3: Can Python be used for web development?
Ans: Yes (Flask, Django)

🔹 4. Variables and Identifiers

📖 Variable

- Used to store data

📖 Identifier

- Name of variable/function

📖 Rules for Identifiers

- Must start with letter or _
- Cannot start with number
- No spaces
- Cannot use keywords

🧪 Example

```
x = 10

name = "Yash"
```

❓ Viva Questions

👉 Q1: What is variable?
Ans: Container to store data

👉 Q2: What is identifier?
Ans: Name of variable/function

👉 Q3: Can variable start with number?
Ans: No

👉 Q4: Can we use keywords as variable?
Ans: No

◈ 5. Declaring and Assigning Values

📖 In Python

- No need to declare type

x = 10

x = "Hello"

👉 Same variable, different types

❓ Viva Questions

👉 Q1: How to declare variable in Python?
Ans: Direct assignment

👉 Q2: What is multiple assignment?

a, b = 10, 20

👉 Q3: Can Python change type of variable?
Ans: Yes (dynamic typing)

◈ 6. Python Operators

📖 Types of Operators

◆ Arithmetic

- +, -, *, /, %, **

◆ Relational

- , <, ==, !=

◆ Logical

- and, or, not

◆ Assignment

- =, +=, -=

❓ Viva Questions

👉 Q1: What are operators?
Ans: Symbols to perform operations

👉 Q2: Difference between == and = ?
Ans:

- = → assignment
- == → comparison

👉 Q3: What is modulus operator?
Ans: Gives remainder (%)

🔷 7. Basic Programming Concepts

📖 Input

x = input("Enter value")

📖 Output

print(x)

📖 Data Types

- int
- float
- string
- bool

❓ Viva Questions

👉 Q1: What is input function?
Ans: Takes user input

👉 Q2: What is output function?
Ans: Displays output

👉 Q3: Default input type?
Ans: String

👉 Q4: How to convert input to int?

int(input())

⚠ VERY IMPORTANT CROSS QUESTIONS

Examiner will go deeper 👇

👉 Q: Why Python is slower than C?
Ans: Because Python is interpreted

👉 Q: What is difference between compiled and interpreted?
Ans:

- Compiled → whole program compiled
- Interpreted → line by line execution

👉 Q: What is dynamically typed language?
Ans: Type decided at runtime

👉 Q: What happens if you don’t indent properly?
Ans: Syntax error

👉 Q: What is memory allocation in Python?
Ans: Automatically managed

🎯 FINAL QUICK REVISION

- Python = interpreted
- Variable = data holder
- Identifier = name
- Dynamic typing
- Operators = perform operations
- Input = input()
- Output = print()

Perfect 👍 — this is **Chapter 2: CONTROL STRUCTURES**, one of the **MOST IMPORTANT viva chapters** 🔥

I'll prepare you like examiner asks:

- 📘 Detailed concepts (simple + clear)
- ❓ All possible viva questions
- ⚠️ Cross-questions (tricky)

📘 CONTROL STRUCTURES (FULL VIVA NOTES)

🔹 1. Python Keywords

📖 Definition

Keywords are **reserved words** in Python that have predefined meaning and cannot be used as identifiers.

💡 Examples

if, else, for, while, break, continue, pass, True, False

❓ Viva Questions

👉 Q1: What are keywords?
Ans: Reserved words with predefined meaning

👉 Q2: Can we use keywords as variable names?
Ans: No

👉 Q3: How to check keywords in Python?

```
import keyword

keyword.kwlist
```

◈ 2. Comments in Python

📖 Definition

Comments are used to **explain code** and are ignored by interpreter.

💡 Types

```
# Single line comment


"""

Multi-line comment

"""
```

❓ Viva Questions

👉 Q1: What is comment?
Ans: Non-executable explanation in code

👉 Q2: Why comments are used?
Ans: For readability and understanding

◈ 3. If Statement

📖 Syntax

```
if condition:

    statement
```

❓ Viva Questions

👉 Q1: What is if statement?
Ans: Decision-making statement

👉 Q2: What type of condition used?
Ans: Boolean expression

◈ 4. If-Else Statement

📖 Syntax

```
if condition:
    statement
else:
    statement
```

❓ Viva Questions

👉 Q1: Difference between if and if-else?
Ans: if → single condition, if-else → two paths

◈ 5. Nested If

📖 Definition

If inside another if.

```
if x > 0:
    if x > 10:
        print("Greater")
```

❓ Viva Questions

👉 Q1: What is nested if?
Ans: If inside another if

👉 Q2: Why used?
Ans: Multiple conditions checking

◈ 6. Indentation

📖 Definition

Spaces used to define block of code in Python.

💡 Example

```
if True:
    print("Hello")
```

❓ Viva Questions

👉 Q1: What is indentation?
Ans: Space used to define block

👉 Q2: Is indentation compulsory?
Ans: Yes

👉 Q3: What happens if indentation is wrong?
Ans: Syntax error

◈ 7. For Loop

📖 Definition

Used to iterate over sequence.

```
for i in range(5):
    print(i)
```

❓ Viva Questions

👉 Q1: What is for loop?
Ans: Loop for iteration

👉 Q2: Where used?
Ans: When number of iterations known

◈ 8. For Loop with Else

📖 Concept

Else executes when loop completes normally.

```
for i in range(3):
    print(i)
else:
    print("Done")
```

❓ Viva Questions

👉 Q1: When else executes in loop?
Ans: When loop ends without break

◈ 9. range() Function

📖 Syntax

range(start, stop, step)

💡 Example

range(1, 10, 2)

❓ Viva Questions

👉 Q1: What is range()?
Ans: Generates sequence of numbers

👉 Q2: Default start value?
Ans: 0

◈ 10. While Loop

📖 Definition

Runs while condition is true.

```
while x < 5:
    x += 1
```

❓ Viva Questions

👉 Q1: Difference between for and while?
Ans:

- for → fixed iterations
- while → condition-based

◈ 11. Break Statement

📖 Definition

Terminates loop immediately.

break

❓ Viva Questions

👉 Q1: What is break?
Ans: Stops loop

◈ 12. Continue Statement

📖 Definition

Skips current iteration.

continue

❓ Viva Questions

👉 Q1: What is continue?
Ans: Skip iteration

◈ 13. Pass Statement

📖 Definition

Does nothing (placeholder).

pass

❓ Viva Questions

👉 Q1: What is pass?
Ans: Empty statement

👉 Q2: Why used?
Ans: When statement required but no code

⚠️ VERY IMPORTANT CROSS QUESTIONS

👉 Q: Difference between break and continue?
Ans:

- break → exit loop
- continue → skip iteration

👉 Q: Can we use else with while?
Ans: Yes

👉 Q: Infinite loop kya hota hai?
Ans: Loop that never ends

👉 Q: What is indentation error?
Ans: Wrong spacing → syntax error

👉 Q: Which loop is faster?
Ans: Depends on condition (no fixed answer)

👉 Q: Can we write loop without range()?
Ans: Yes (using list, string, etc.)

🎯 QUICK REVISION (LAST MINUTE)

- Keywords = reserved words
- If = decision
- Loop = repetition

- For = fixed iteration
- While = condition-based
- Break = stop
- Continue = skip
- Pass = do nothing
- Indentation = mandatory

📓 DATA TYPES & DATA STRUCTURES (FULL VIVA NOTES)

◆ 1. STRINGS

◈ String Basics

- String = sequence of characters
- Immutable (cannot change)

```
s = "Hello"
```

◈ Indexing and Slicing

- Index starts from 0

```
s[0]      # H
s[0:3]    # Hel
s[-1]     # o
```

◈ String Operations

- Concatenation → +
- Repetition → *

```
"Hi" + "Hello"
"Hi"*3
```

◈ String Functions

- len()
- upper()
- lower()
- replace()
- find()

❓ VIVA QUESTIONS (STRING)

👉 What is string?
Ans: Sequence of characters

👉 Is string mutable?
Ans: No (immutable)

👉 What is slicing?
Ans: Extract part of string

👉 Difference between + and *?
Ans: + → join, * → repeat

◆ 2. LISTS

◈ List Basics

- Ordered, mutable collection

l = [1,2,3]

◈ Indexing and Slicing

l[0]

l[1:3]

◈ List Operations

- +, *, in

◈ Adding Elements

- append() → end
- insert() → specific position

◈ Removing Elements

- remove()
- pop()
- del

◈ Built-in List Methods

- sort()
- reverse()
- count()
- index()

❓ VIVA QUESTIONS (LIST)

👉 What is list?
Ans: Mutable ordered collection

👉 Difference between list and string?
Ans: List mutable, string immutable

👉 append() vs insert()?
Ans:

- append → end
- insert → position

◆ 3. TUPLES

◈ Tuple Basics

- Immutable collection

t = (1,2,3)

◈ Accessing Elements

t[0]

◈ Indexing and Slicing

Same as list

◈ Tuple Methods

- count()
- index()

◈ Advantages of Tuples

- Faster
- Secure (no modification)

❓ VIVA QUESTIONS (TUPLE)

👉 What is tuple?
Ans: Immutable collection

👉 Why tuple faster than list?
Ans: No modification allowed

👉 Can we modify tuple?
Ans: No

◆ 4. SETS

◈ Set Basics

- Unordered
- No duplicate values

s = {1,2,3}

◈ Creating Sets

set([1,2,3])

◈ Adding & Updating

- add()
- update()

◈ Removing Elements

- remove()
- discard()

◈ discard() vs remove()

remove()	discard()
Error if not found	No error

◈ Frozen Set

- Immutable set

❓ VIVA QUESTIONS (SET)

👉 What is set?
Ans: Collection of unique elements

👉 Are sets ordered?
Ans: No

👉 remove vs discard?
Ans: discard safe

◆ 5. DICTIONARY

◈ Dictionary Basics

- Key-value pairs

d = {"name":"Yash", "age":20}

◈ Creating Dictionary

dict()

◈ Accessing Values

d["name"]

◈ Adding & Deleting

d["city"] = "Vadodara"

del d["age"]

◈ Properties of Keys

- Unique
- Immutable

◈ Dictionary Methods

- keys()
- values()
- items()
- get()

❓ VIVA QUESTIONS (DICTIONARY)

👉 What is dictionary?
Ans: Key-value data structure

👉 Can keys be duplicate?
Ans: No

👉 Can values be duplicate?
Ans: Yes

👉 Why dictionary fast?
Ans: Uses hashing

⚠ VERY IMPORTANT COMPARISON QUESTIONS

🔥 List vs Tuple

List	Tuple
Mutable	Immutable
Slow	Fast

🔥 List vs Set

List	Set
Ordered	Unordered
Duplicate allowed	No duplicates

🔥 Tuple vs Set

Tuple	Set
Ordered	Unordered
Immutable	Mutable

🔥 Dictionary vs List

Dictionary	List
Key-value	Index-based

⚠ TRICKY QUESTIONS (VERY IMPORTANT)

👉 Q: Which data type allows duplicates?
Ans: List, Tuple

👉 Q: Which is fastest?
Ans: Tuple

👉 Q: Which uses key-value?
Ans: Dictionary

👉 Q: Can set contain list?
Ans: No (list mutable)

👉 Q: Can tuple contain list?
Ans: Yes

👉 Q: What is hashing?
Ans: Technique for fast lookup

🎯 **FINAL QUICK REVISION**

- String → immutable
- List → mutable
- Tuple → immutable
- Set → unique values
- Dictionary → key-value

📓 **FUNCTIONS (FULL VIVA NOTES)**

🔷 1. Introduction to Functions

📖 Definition

Function is a **block of reusable code** that performs a specific task.

💡 Why use functions?

- Code reuse
- Reduce repetition
- Better readability

📦 Example

```
def greet():
    print("Hello")
```

❓ Viva Questions

👉 What is function?
Ans: Block of reusable code

👉 Why functions are used?
Ans: To avoid repetition and improve readability

🔷 2. Function Definition and Calling

📖 Definition

- Define using def
- Call using function name

📦 Example

```
def greet():
    print("Hello")


greet()   # function call
```

❓ Viva Questions

👉 How to define function?
Ans: Using def keyword

👉 How to call function?
Ans: Using function name

◈ 3. Types of Functions

📖 Types:

1. Built-in functions
    - len(), print()
2. User-defined functions
    - Created by programmer
3. Lambda functions (anonymous)

❓ Viva Questions

👉 What are types of functions?
Ans: Built-in, user-defined, lambda

👉 What is lambda function?
Ans: Anonymous one-line function

◈ 4. Function Syntax

```
def function_name(parameters):
    statements
    return value
```

❓ Viva Questions

👉 What is syntax of function?
Ans: def + name + parameters + return

👉 Is return compulsory?
Ans: No

◈ 5. Parameters and Arguments

📖 Difference

- Parameter → in definition
- Argument → in function call

🧪 Example

```
def add(a, b):   # parameters
    return a+b

add(2,3)         # arguments
```

❓ Viva Questions

👉 Difference between parameter and argument?
Ans:

- Parameter → definition
- Argument → calling

🔷 6. Default Arguments

📖 Definition

Default value given to parameter

🧪 Example

```
def greet(name="User"):
    print(name)
```

❓ Viva Questions

👉 What is default argument?
Ans: Predefined value

👉 When used?
Ans: When argument not provided

◈ 7. Keyword Arguments

📖 Definition

Arguments passed using parameter name

▯ Example

add(a=2, b=3)

❓ Viva Questions

👉 What is keyword argument?
Ans: Argument with name

👉 Advantage?
Ans: Order not important

◈ 8. Positional Arguments

📖 Definition

Arguments passed in order

▯ Example

add(2,3)

❓ Viva Questions

👉 What is positional argument?
Ans: Order-based argument

👉 What if order changes?
Ans: Wrong result

◈ 9. Arbitrary Arguments (*args, **kwargs)

◆ *args (non-keyword arguments)

- Multiple values in tuple

```
def add(*nums):
    print(nums)
```

◆ **kwargs (keyword arguments)

- Multiple values in dictionary

```
def info(**data):
    print(data)
```

❓ Viva Questions

👉 What is *args?
Ans: Multiple arguments in tuple

👉 What is **kwargs?
Ans: Multiple arguments in dictionary

👉 Difference?
Ans:

- *args → tuple
- **kwargs → dictionary

◈ 10. Pass by Value vs Reference

📖 Concept

👉 Immutable (int, string) → pass by value
👉 Mutable (list, dict) → pass by reference

▯ Example

```
def change(x):
    x = 10
```

❓ Viva Questions

👉 What is pass by value?
Ans: Copy of value passed

👉 What is pass by reference?
Ans: Original value modified

👉 Python uses which?
Ans: Both (depends on data type)

⚠ VERY IMPORTANT CROSS QUESTIONS

👉 Q: Can function return multiple values?
Ans: Yes (tuple)

👉 Q: What is recursion?
Ans: Function calling itself

👉 Q: What happens if no return?
Ans: Returns None

👉 Q: Can we call function inside function?
Ans: Yes

👉 Q: Difference between parameter and argument?
Ans: Already explained (VERY IMPORTANT)

👉 Q: What is scope of variable?
Ans:

- Local → inside function
- Global → outside function

👉 Q: What is global keyword?
Ans: Used to access global variable

🎯 FINAL QUICK REVISION

- Function = reusable code
- def → define
- call → function_name()
- Parameter ≠ Argument
- *args → tuple
- **kwargs → dictionary
- Default argument → optional
- Return → gives output

📓 OBJECT ORIENTED PROGRAMMING (FULL VIVA NOTES)

🔷 1. OOP Concepts

📖 Definition

OOP is a programming approach based on **objects and classes**.

💡 Key Concepts

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

❓ Viva Questions

👉 What is OOP?
Ans: Programming based on objects and classes

👉 Why OOP is used?
Ans: Code reuse, security, modularity

◈ 2. Principles of OOP

📖 4 Main Principles:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

❓ Viva Questions

👉 What are pillars of OOP?
Ans: Encapsulation, Inheritance, Polymorphism, Abstraction

◈ 3. Class and Object

📖 Class

- Blueprint/template

📖 Object

- Instance of class

▯ Example

```
class Student:
    pass

s1 = Student()
```

❓ Viva Questions

👉 What is class?
Ans: Blueprint

👉 What is object?
Ans: Instance of class

👉 What is self?
Ans: Reference to current object

◈ 4. Encapsulation

📖 Definition

Wrapping data and methods into one unit

💡 Example

- Using private variables

```
class A:
    def __init__(self):
        self.__x = 10
```

❓ Viva Questions

👉 What is encapsulation?
Ans: Data hiding

👉 Why used?
Ans: Security

◈ 5. Inheritance

📖 Definition

Child class inherits properties of parent

▯ Example

```
class Parent:
```

```
    pass

class Child(Parent):
    pass
```

❓ Viva Questions

👉 What is inheritance?
Ans: Code reuse

👉 Advantage?
Ans: Reduces redundancy

🔹 6. Single Inheritance

- One parent → one child

❓ Viva Questions

👉 Define single inheritance
Ans: One parent one child

🔹 7. Multiple Inheritance

- One child → multiple parents

❓ Viva Questions

👉 Define multiple inheritance
Ans: One child, multiple parents

👉 Problem?
Ans: Diamond problem

🔹 8. Multilevel Inheritance

- Grandparent → Parent → Child

❓ Viva Questions

👉 Define multilevel inheritance
Ans: Chain inheritance

🔷 9. Polymorphism

📖 Definition

Same function, different behavior

💡 Types

- Method overloading
- Method overriding

🧪 Example

print(len("Hello"))

print(len([1,2,3]))

❓ Viva Questions

👉 What is polymorphism?
Ans: Same name, different behavior

👉 Types?
Ans: Overloading, overriding

🔷 10. Data Abstraction

📖 Definition

Hiding internal details and showing only necessary parts

❓ Viva Questions

👉 What is abstraction?
Ans: Hide complexity

◈ 11. Abstract Classes

📖 Definition

Class that cannot be instantiated directly

🧪 Example

```
from abc import ABC


class A(ABC):
    pass
```

❓ Viva Questions

👉 What is abstract class?
Ans: Cannot create object

👉 Why used?
Ans: To define structure

⚠ VERY IMPORTANT CROSS QUESTIONS

👉 Q: Difference between class and object?
Ans:

- Class → blueprint
- Object → instance

👉 Q: Encapsulation vs Abstraction?
Ans:

- Encapsulation → data hiding
- Abstraction → hide complexity

👉 Q: Inheritance vs Polymorphism?
Ans:

- Inheritance → reuse
- Polymorphism → flexibility

👉 Q: What is constructor?
Ans: Special method __init__()

👉 Q: What is method?
Ans: Function inside class

👉 Q: What is self keyword?
Ans: Current object reference

👉 Q: Can Python support multiple inheritance?
Ans: Yes

👉 Q: What is MRO?
Ans: Method Resolution Order

👉 Q: What is overriding?
Ans: Redefining parent method

👉 Q: What is overloading?
Ans: Same function, different parameters

🎯 **FINAL QUICK REVISION**

- OOP = class + object
- Encapsulation = data hiding
- Inheritance = reuse
- Polymorphism = multiple behavior

- Abstraction = hide complexity
- Abstract class = no object

📓 EXCEPTION HANDLING (FULL VIVA NOTES)

🔷 1. Introduction to Exceptions

📖 Definition

Exception is a **runtime error** that occurs during program execution.

💡 Examples

- Division by zero
- File not found
- Invalid input

▫️ Example

x = 10 / 0 # error

❓ Viva Questions

👉 What is exception?
Ans: Runtime error

👉 Difference between error and exception?
Ans:

- Error → compile time
- Exception → runtime

👉 Why exception handling is needed?
Ans: To prevent program crash

🔷 2. Try-Except Block

📖 Definition

Used to handle exceptions safely.

□ Syntax

```
try:
    risky code
except:
    handle error
```

□ Example

```
try:
    x = 10/0
except:
    print("Error handled")
```

❓ Viva Questions

👉 What is try block?
Ans: Contains risky code

👉 What is except block?
Ans: Handles error

👉 Can we use multiple except?
Ans: Yes

◈ 3. Raising Exceptions

📖 Definition

Manually generating exception using raise.

□ Example

```
raise ValueError("Invalid input")
```

❓ Viva Questions

👉 What is raise?
Ans: Used to create exception manually

👉 Why used?
Ans: To control program flow

◈ 4. Assertions

📖 Definition

Used to check condition during debugging.

▯ Example

```
assert x > 0
```

❓ Viva Questions

👉 What is assertion?
Ans: Debugging check

👉 When assertion fails?
Ans: Raises AssertionError

◈ 5. Try with Else

📖 Concept

Else executes if **no exception occurs**

▯ Example

```
try:
    x = 10/2
except:
    print("Error")
else:
```

```
    print("Success")
```

❓ Viva Questions

👉 When else executes?
Ans: When no exception

◈ 6. Finally Block

📖 Definition

Always executes (whether error occurs or not)

▯ Example

```
try:
    x = 10/0
except:
    print("Error")
finally:
    print("Always runs")
```

❓ Viva Questions

👉 What is finally block?
Ans: Always executes

👉 Why used?
Ans: Cleanup (close file, release memory)

⚠ VERY IMPORTANT CROSS QUESTIONS

👉 Q: Can we write try without except?
Ans: Yes (with finally)

👉 Q: Can we write except without try?
Ans: No

👉 Q: Order of execution?
Ans:

1. try
2. except
3. else (if no error)
4. finally

👉 Q: Difference between finally and else?
Ans:

- else → runs if no error
- finally → always runs

👉 Q: What happens if exception not handled?
Ans: Program crashes

👉 Q: Can we catch specific exception?
Ans: Yes

except ZeroDivisionError:

👉 Q: What is base class of exceptions?
Ans: Exception

👉 Q: What is runtime error?
Ans: Error during execution

🎯 **FINAL QUICK REVISION**

- Exception = runtime error
- try = risky code

- except = handle error
- else = no error
- finally = always runs
- raise = manual error
- assert = debugging

📓 FILE HANDLING (FULL VIVA NOTES)

🔹 1. Introduction to File Handling

📖 Definition

File handling means **reading from and writing to files** stored on disk.

💡 Why needed?

- Store data permanently
- Read large data
- Data sharing

❓ Viva Questions

👉 What is file handling?
Ans: Reading/writing data from file

👉 Why file handling used?
Ans: For permanent storage

👉 Difference between file and variable?
Ans:

- Variable → temporary (RAM)
- File → permanent (disk)

🔹 2. File Opening Modes

📖 Syntax

```
file = open("file.txt", "mode")
```

📖 Modes

Mode	Meaning
r	Read
w	Write (overwrite)
a	Append
r+	Read + Write

❓ Viva Questions

👉 What is open()?
Ans: Opens file

👉 Difference between w and a?
Ans:

- w → overwrite
- a → append

👉 What happens if file not exist in w mode?
Ans: File created

🔳 3. Reading Files

◆ read()

- Reads entire file

```
f.read()
```

◆ readline()

- Reads one line

◆ readlines()

- Returns list of lines

❓ Viva Questions

👉 Difference between read() and readline()?
Ans:

- read() → full file
- readline() → one line

👉 What does readlines() return?
Ans: List

◈ 4. Writing Files

◆ write()

- Writes string

◆ writelines()

- Writes list

▯ Example

```
f.write("Hello")
```

❓ Viva Questions

👉 Difference between write and writelines()?
Ans:

- write → string
- writelines → list

◈ 5. Closing Files

📖 Definition

Close file after use

f.close()

❓ Viva Questions

👉 Why close file?
Ans: Free memory/resources

🔹 6. with Statement

📖 Definition

Automatically closes file

💻 Example

```
with open("file.txt","r") as f:
    data = f.read()
```

❓ Viva Questions

👉 Why with statement used?
Ans: Auto close file

👉 Advantage?
Ans: Safe and clean code

🔹 7. Exception Handling in Files

📖 Definition

Handle file errors (like file not found)

💻 Example

```
try:
    f = open("file.txt")
```

```
except:
    print("Error")
```

❓ Viva Questions

👉 Why exception handling in file?
Ans: To avoid crash

👉 Example error?
Ans: FileNotFoundError

⚠ VERY IMPORTANT CROSS QUESTIONS

👉 Q: What happens if file not closed?
Ans: Memory leak / resource issue

👉 Q: Default mode of open()?
Ans: 'r'

👉 Q: Which mode creates file?
Ans: 'w'

👉 Q: Can we read file in write mode?
Ans: No

👉 Q: Difference between r+ and w+?
Ans:

- r+ → no overwrite
- w+ → overwrite

👉 Q: Which method returns list?
Ans: readlines()

👉 Q: Which is safer: open() or with?
Ans: with

👉 Q: What is file pointer?
Ans: Position inside file

👉 Q: How to move pointer?
Ans: seek()

👉 Q: What is tell()?
Ans: Returns current position

🎯 FINAL QUICK REVISION

- File = permanent storage
- open() → open file
- read() → full file
- write() → write text
- close() → close file
- with → auto close
- Exception → handle errors

📘 MODULES & PACKAGES (FULL VIVA NOTES)

🔷 1. Introduction to Modules

📖 Definition

Module = **Python file (.py) containing functions, variables, classes**

💡 Example

file: math_utils.py

```
def add(a,b):
    return a+b
```

❓ Viva Questions

👉 What is module?
Ans: Python file containing code

👉 Why modules used?
Ans: Code reuse

👉 Extension of module?
Ans: .py

◈ 2. Working with Modules

📖 Importing Modules

```
import math

math.sqrt(16)
```

📖 Types of Import

```
import module

from module import function

import module as alias
```

❓ Viva Questions

👉 What is import?
Ans: Used to use module

👉 Difference:

- import math vs from math import sqrt
  Ans:
- First → full module
- Second → specific function

👉 What is alias?
Ans: Short name

🔹 3. Packages in Python

📖 Definition

Package = **collection of modules (folder with init.py)**

💡 Example Structure

```
mypackage/
  module1.py
  module2.py
```

❓ Viva Questions

👉 What is package?
Ans: Collection of modules

👉 Difference module vs package?
Ans:

- Module → file
- Package → folder

🔹 4. Modular Programming

📖 Definition

Breaking program into small modules

💡 Advantages

- Easy maintenance
- Code reuse
- Better readability

❓ Viva Questions

👉 What is modular programming?
Ans: Divide program into modules

👉 Advantage?
Ans: Easy debugging

◈ 5. Popular Python Libraries

📖 Examples

- NumPy → numerical
- Pandas → data analysis
- Matplotlib → graphs
- Requests → API

❓ Viva Questions

👉 What is library?
Ans: Collection of modules

👉 Difference library vs module?
Ans:

- Module → single file
- Library → collection

◈ 6. PyCharm IDE

📖 Definition

IDE = Integrated Development Environment

💡 Features

- Code editor
- Debugging
- Auto-complete

❓ Viva Questions

👉 What is IDE?
Ans: Tool for coding

👉 Example?
Ans: PyCharm

🔷 7. Git Integration

📖 Definition

Git = Version control system

💡 Use

- Track changes
- Collaboration

❓ Viva Questions

👉 What is Git?
Ans: Version control system

👉 Why used?
Ans: Track code changes

🔷 8. PyTest

📖 Definition

Framework used for testing Python code

❓ Viva Questions

👉 What is PyTest?
Ans: Testing framework

👉 Why testing needed?
Ans: Check correctness

◈ 9. Database Connectivity (MySQL, MongoDB)

📖 Definition

Connecting Python with database

💡 Types

- MySQL → Relational DB
- MongoDB → NoSQL DB

▯ Example

```
import sqlite3

conn = sqlite3.connect("db.db")
```

❓ Viva Questions

👉 What is database?
Ans: Data storage system

👉 Difference MySQL vs MongoDB?
Ans:

- MySQL → table-based
- MongoDB → document-based

👉 Why connect Python to DB?
Ans: Store/retrieve data

⚠ VERY IMPORTANT CROSS QUESTIONS

👉 Q: Difference between module and package?
Ans:

- Module → file
- Package → folder

👉 Q: What is **init**.py?
Ans: Makes folder a package

👉 Q: Difference between import and from?
Ans:

- import → full module
- from → specific function

👉 Q: What is pip?
Ans: Package installer

👉 Q: What is virtual environment?
Ans: Isolated environment

👉 Q: What is version control?
Ans: Track code changes

👉 Q: What is testing?
Ans: Checking code correctness

👉 Q: SQL vs NoSQL?
Ans:

- SQL → structured
- NoSQL → flexible

🎯 **FINAL QUICK REVISION**

- Module = .py file
- Package = folder
- import = use module
- Library = collection

- Git = version control
- PyTest = testing
- DB = data storage

📓 REST API & WEB DEVELOPMENT (FULL VIVA NOTES)

🔹 1. Introduction to REST API

📖 Definition

REST API = **communication system between client and server using HTTP**

💡 Example

- Browser (client) → request → Server → response

❓ Viva Questions

👉 What is REST API?
Ans: Communication between client and server

👉 What is client?
Ans: User side (browser/app)

👉 What is server?
Ans: Provides data

🔹 2. HTTP Methods

📖 Types

Method	Use
GET	Fetch data
POST	Create data
PUT	Update data
DELETE	Remove data

❓ Viva Questions

👉 What is GET?
Ans: Retrieve data

👉 Difference GET vs POST?
Ans:

- GET → fetch
- POST → send data

◈ 3. HTTP Status Codes

📖 Types

Code	Meaning
200	Success
300	Redirect
404	Not found
500	Server error

❓ Viva Questions

👉 What is 200?
Ans: Success

👉 What is 404?
Ans: Page not found

👉 What is 500?
Ans: Server error

◈ 4. API Error Handling

📖 Definition

Handling errors in API

❓ Viva Questions

👉 Why error handling needed?
Ans: To avoid crashes

🔹 5. API Versioning

📖 Definition

Managing API updates

💡 Example

/api/v1/users

❓ Viva Questions

👉 What is API versioning?
Ans: Managing changes

🔹 6. REST API Best Practices

- Use proper HTTP methods
- Use status codes
- Use meaningful URLs
- Secure API

❓ Viva Questions

👉 Best practice?
Ans: Proper methods + security

🔹 7. API Testing

📖 Tools

- cURL
- Postman

- requests (Python library)

❓ Viva Questions

👉 What is Postman?
Ans: API testing tool

👉 What is requests library?
Ans: Python library for API

◈ 8. Flask Framework

📖 Definition

Lightweight Python web framework

▯ Example

```
from flask import Flask

app = Flask(__name__)
```

❓ Viva Questions

👉 What is Flask?
Ans: Lightweight web framework

👉 Why used?
Ans: Build APIs

◈ 9. FastAPI Framework

📖 Definition

Modern, fast API framework

❓ Viva Questions

👉 What is FastAPI?
Ans: High-performance API framework

👉 Difference Flask vs FastAPI?
Ans:

- Flask → simple
- FastAPI → faster

◈ 10. API Security

📖 Types

- API Keys
- JWT (JSON Web Token)
- OAuth
- HTTPS

❓ Viva Questions

👉 What is JWT?
Ans: Token for authentication

👉 Why HTTPS used?
Ans: Secure communication

◈ 11. Scalable API Design

📖 Definition

API that handles large users

❓ Viva Questions

👉 What is scalability?
Ans: Handle more users

◈ 12. REST Architecture

📖 Principles

- Stateless

- Client-server
- Cacheable

❓ Viva Questions

👉 What is stateless?
Ans: Server does not store client data

👉 What is REST architecture?
Ans: Rules for API design

⚠️ VERY IMPORTANT CROSS QUESTIONS

👉 Q: Difference between API and REST API?
Ans:

- API → general
- REST API → follows REST rules

👉 Q: GET vs POST?
Ans:

- GET → read
- POST → create

👉 Q: Why REST stateless?
Ans: Better scalability

👉 Q: What is JSON?
Ans: Data format

👉 Q: What is endpoint?
Ans: URL of API

👉 Q: Flask vs FastAPI?
Ans:

- Flask → simple
- FastAPI → faster, modern

👉 Q: What is authentication vs authorization?
Ans:

- Auth → who you are
- Authorization → what you can access

🎯 **FINAL QUICK REVISION**

- REST API = client-server communication
- GET = read, POST = create
- 200 = success, 404 = not found
- Flask = simple API
- FastAPI = fast API
- JWT = security
- Stateless = no memory